OPERATING SYSTEM DESIGN

S Thenmozhi

Department of Computer Applications

OPERATING SYSTEM DESIGN

PROCESS & THREAD MANAGMENT

S Thenmozhi
Department of Computer Applications

Deadlock Avoidance – Bankers Algorithm

Deadlock avoidance of Resources with Multiple instance can be solved using Bankers algorithm

Data structures for Bankers Algorithm

Let n = number of processes, and m = number of resources types

- **Available***:* Vector of length m. If available [j] = k, there are k instances of resource type R_j available
- **Max***:* n *x* m matrix. If *Max* [i,j] = k, then process P_i may request at most k instances of resource type R_j
- **Allocation***:* n x m matrix. If Allocation[i,j] = k then P_i is currently allocated k instances of R_j
- **Need***:* n x m matrix. If *Need*[i,j] = k, then P_i may need k more instances of R_j to complete its task

$$Need\ [i,j] = Max[i,j] - Allocation\ [i,j]$$

Deadlock Avoidance – Bankers Algorithm

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively. Initialize:

   Work = *Available*

   Finish [*i*] = *false* for *i* = 0, 1, …, *n*- 1

2. Find an *i* such that both:

   (a) *Finish* [*i*] = *false*

   (b) $Need_i \leq Work$

   If no such *i* exists, go to step 4

3. $Work = Work + Allocation_i$
   Finish[*i*] = *true*
   go to step 2

4. If *Finish* [*i*] == true for all *i*, then the system is in a safe state

Deadlock Avoidance – Bankers Algorithm

- 5 processes P_0 through P_4;
- 3 resource types: *A* (10 instances), *B* (5instances), and *C* (7 instances)

Snapshot at time T_0:

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Deadlock Avoidance – Bankers Algorithm

- The content of the matrix *Need* is defined to be

Max – Allocation

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

	Need
	A B C
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

- The system is in a safe state since the sequence **< P1, P3, P4, P0, P2 >** satisfies safety criteria

m=3, n=5 — Step 1 of Safety Algo

Work = Available

Work = | 3 | 3 | 2 |

	0	1	2	3	4
Finish =	false	false	false	false	false

For i = 0 — Step 2

$Need_0$ = 7, 4, 3 (7,4,3 vs 3,3,2)

Finish [0] is false and $Need_0$ > Work

So P_0 must wait — But Need ≤ Work

For i = 1 — Step 2

$Need_1$ = 1, 2, 2 (1,2,2 vs 3,3,2)

Finish [1] is false and $Need_1$ < Work

So P_1 must be kept in safe sequence

Step 3

3, 3, 2 + 2, 0, 0

Work = Work + $Allocation_1$

Work =	A	B	C
	5	3	2

	0	1	2	3	4
Finish =	false	true	false	false	false

For i = 2 — Step 2

$Need_2$ = 6 , 0, 0 (6, 0, 0 vs 5,3, 2)

Finish [2] is false and $Need_2$ > Work

So P_2 must wait

For i=3 — Step 2

$Need_3$ = 0, 1, 1

0, 1, 1 vs 5, 3, 2

Finish [3] = false and $Need_3$ < Work

So P_3 must be kept in safe sequence

Step 3

5, 3, 2 + 2, 1, 1

Work = Work + $Allocation_3$

Work =	A	B	C
	7	4	3

	0	1	2	3	4
Finish =	false	true	false	true	false

For i = 4 — Step 2

$Need_4$ = 4, 3, 1

4, 3, 1 vs 7, 4, 3

Finish [4] = false and $Need_4$ < Work

So P_4 must be kept in safe sequence

Step 3

7, 4, 3 + 0, 0, 2

Work = Work + $Allocation_4$

Work =	A	B	C
	7	4	5

	0	1	2	3	4
Finish =	false	true	false	true	true

For i = 0 — Step 2

$Need_0$ = 7, 4, 3

7, 4, 3 vs 7, 4, 5

Finish [0] is false and Need < Work

So P_0 must be kept in safe sequence

Step 3

7, 4, 5 + 0, 1 , 0

Work = Work + $Allocation_0$

Work =	A	B	C
	7	5	5

	0	1	2	3	4
Finish =	true	true	false	true	true

For i = 2 — Step 2

$Need_2$ = 6 , 0, 0

6, 0, 0 vs 7, 5, 5

Finish [2] is false and $Need_2$ < Work

So P_2 must be kept in safe sequence

Step 3

7, 5, 5 + 3, 0, 2

Work = Work + $Allocation_2$

Work =	A	B	C
	10	5	7

	0	1	2	3	4
Finish =	true	true	true	true	true

Step 4

Finish [i] = true for 0 ≤ i ≤ n

Hence the system is in Safe state

The safe sequence is P_1, P_3 , P_4 , P_0, P_2

Deadlock Avoidance – Bankers Algorithm

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	1	1	0	0	2	1	0	1	3	1	0
P_1	1	4	4	1	1	6	5	2				
P_2	1	3	6	5	2	3	6	6				
P_3	0	6	3	2	0	6	5	2				
P_4	0	0	1	4	0	6	5	6				

Deadlock Avoidance – Bankers Algorithm

	Allocation Matrix (N0 of the allocated resources By a process)				Max Matrix Max resources that may be used by a process				Available Matrix Not Allocated Resources			
	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	1	1	0	0	2	1	0	1	5	2	0
P_1	1	2	3	1	1	6	5	2				
P_2	1	3	6	5	2	3	6	6				
P_3	0	6	3	2	0	6	5	2				
P_4	0	0	1	4	0	6	5	6				
Total	**2**	**12**	**14**	**12**								

Deadlock Avoidance – Bankers Algorithm

- Snapshot at time T_0:

	Allocation	*Max*	*Available*	*Need*
	A B C D	*A B C D*	*A B C D*	*A B C D*
P_0	0 0 1 2	0 0 1 2	1 5 2 0	0 0 0 0
P_1	1 0 0 0	1 7 5 0		0 7 5 0
P_2	1 3 5 4	2 3 5 6		1 0 0 2
P_3	0 6 3 2	0 6 5 2		0 0 2 0
P_4	0 0 1 4	0 6 5 6		0 6 4 2

Safe Sequence: <P0, P2,P3,P4,P1>

Deadlock Avoidance – Bankers Algorithm

- If a request from process P1 arrives with (0,4,2,0) can the request be granted immediately?
- Step1: If $Request_i <= Need_i$
- Step 2: *if* $Request_i <= Available$
- *Step3:* $Available = Available - Request$

  $Allocation_i = Allocation_i + Request_i$

  $Need_i = Need_i - Request_i$

If it is granted, Generate the safe sequence.

Deadlock Avoidance – Bankers Algorithm

- If a request from process P1 arrives with (0,4,2,0) can the request be granted immediately?
- Request of P1 is 0 4 2 0
- Need of P1 is Max-Alloc = 1 7 5 0 – 1 0 0 0 = 0 7 5 0
- Step1: If $Request_i$ <= $Need_i$ 0 4 2 0 < 0 7 5 0 - T
- Step 2: *if $Request_i$ <= Available*
- 0 4 2 0 <= 1 5 2 0 - T
- *Step3: Available = Available – Request*
  - *1 5 2 0 – 0 4 2 0=1 1 0 0*
    $Allocation_i$ = $Allocation_i$ + $Request_i$
  - *1 0 0 0 + 0 4 2 0 = 1 4 2 0*
    $Need_i$ = $Need_i$– $Request_i$
  - 0 7 5 0 - 0 4 2 0 = 0 3 3 0

	Allocation	Max	Available	Need
	A B C D	A B C D	A B C D	A B C D
P_0	0 0 1 2	0 0 1 2	1 5 2 0	0 0 0 0
P_1	1 0 0 0	1 7 5 0		0 7 5 0
P_2	1 3 5 4	2 3 5 6		1 0 0 2
P_3	0 6 3 2	0 6 5 2		0 0 2 0
P_4	0 0 1 4	0 6 5 6		0 6 4 2

Deadlock Avoidance – Bankers Algorithm

This results in the value of Available being (1, 1, 0, 0).

Now Apply the Bankers algorithm and generate the safe sequence.

On ordering of processes they can finish in the order of P0, P2, P3, P1, and P4.

THANK YOU

S Thenmozhi

Department of Computer Applications

thenmozhis@pes.edu

+91 80 6666 3333 Extn 393